

UNIVERSIDAD SAN PABLO CEU

ESCUELA POLITÉCNICA SUPERIOR

INGENIERÍA BIOMÉDICA



PROJECT REPORT

Practice: *Pharmacy Inventory System*

DataBases

Paola Amor Romero, Alba Casado, Andrea Contreras y Alex Martínez

INDEX:

1. Introduction	4
2. Project Statement and Objectives	4
3. System Structure	5
3.1 Package Organization	
3.2 General System Architecture	
4. Model	5
4.1 Entity-Relationship Model	5
4.2 POJO Classes	6
User Class	
Role Class	
Medication Class	
Pharmacy Class	
Inventory Class	
Purchase Class	
Municipality Class	
Order Class	
Supplier Class	
Client Class	
History Class	
Global Stats Class	
MedicationStats Class	
4.3 Relationships Between Entities	7
4.4 Role System	8
Administrator Role	
Pharmacist Role	
4.5 Authentication Subsystem	8
Login	
Password Encryption with BCrypt	
5. Database Managers	9
5.1 ConnectionManager and SQLManager	

5.2 JDBC Managers and Main Functions	
JDBCMedicationManager	
JDBCInventoryManager	
JDBCOrdersManager	
JDBCPharmacyManager	
JDBCPurchaseManager	
JDBCSupplierManager	
JDBCHistoryManager	
JDBCReportManager	
5.3 JPA Managers	
JPAUserManager	
JPARoleManager	
5.4 XML Manager (JAXBManager)	
6. Exceptions and Validations	10
7. User Interface	10
7.1 Main.java	
7.2 Administrator Menu	
7.3 Pharmacist Menu	
8. Testing and Validation	11
8.1 CRUD Testing	
8.2 Login Testing	
8.3 XML Testing	
9. Difficulties and Solved Problems	11
9.1 JDBC Problems	
9.2 JPA Problems	
9.3 BCrypt Problems	
9.4 XML Problems	
10. Conclusions and Bibliography	17
11. UML	17
11.1 Class Diagram and Package Relationships	

1. Introduction

This project consists of a pharmacy management system based on object-oriented programming. The application is connected to a relational database and allows the management of information related to pharmacies, medications, suppliers, inventories, purchases and orders.

The project has been organized using a modular structure based on packages, managers and interfaces. In addition, technologies such as JDBC, JPA and XML have been incorporated in order to manage the database and the authentication subsystem.

2. Project Statement and Objectives

The main objective of this project is to develop an application capable of storing and managing pharmacy-related information through a relational database.

The system must allow CRUD operations on different entities, implement a login subsystem with administrator and pharmacist roles, and support XML import/export functionalities using JAXB.

3. System Structure

The database has been divided into different packages in order to separate the functionalities of the system and improve code organization.

3.1 Package organization

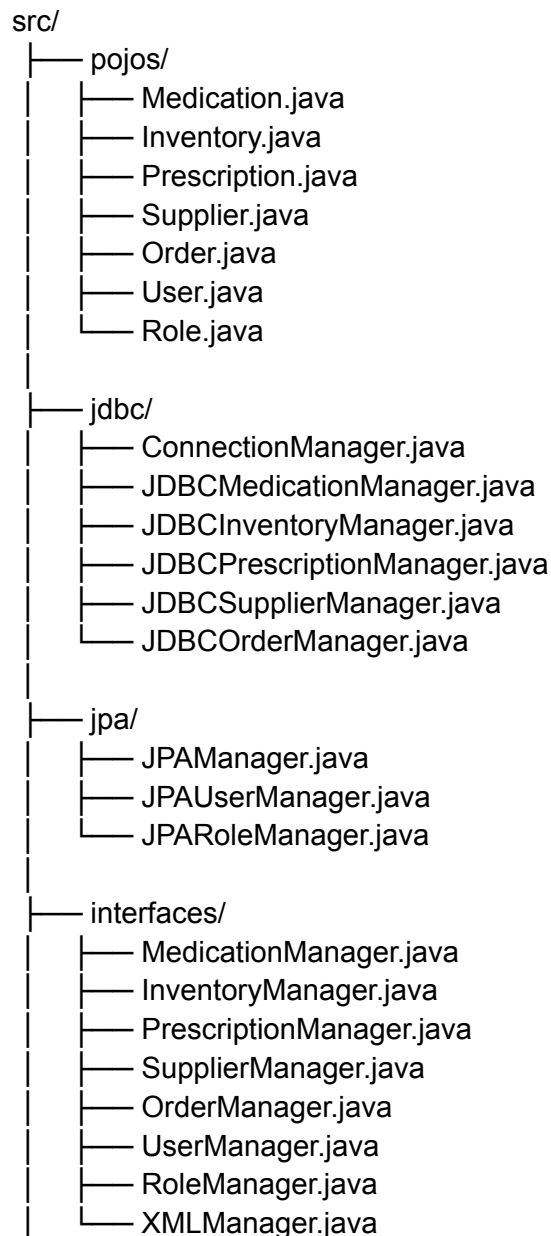
The main packages are:

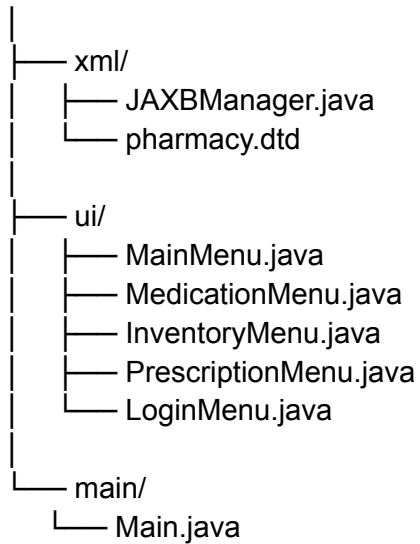
- **pojos**: contains the entity classes of the system, such as Medication, Pharmacy, Inventory, Purchase, Order, Supplier, Client, Municipality, User and Role.
- **jdbc**: contains the JDBC managers responsible for CRUD operations and database interaction.
- **jpa**: contains the authentication subsystem and user-role management using JPA.
- **interfaces**: contains the interfaces implemented by the managers of the application.
- **xml**: contains the XML subsystem developed with JAXB.
- **main**: contains the Main class and the menu-based user interface.

3.2 General System Architecture

The system follows an architecture that divides the database in modules where different packages interact with each other.

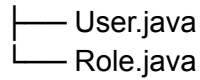
Firstly the user interacts with the database through the UI and the main where the menu is implemented. Then the managers are responsible for communication with the database and producing the different operations of the system. The pojo classes represent the entities stored in the database where the XML subsystem manages data export and import functions.



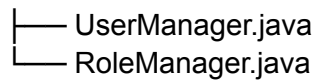


Classes that are needed for the JPA (login subsystem):

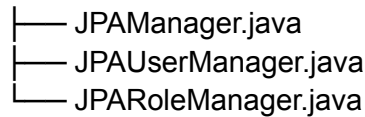
pojos/



interfaces/

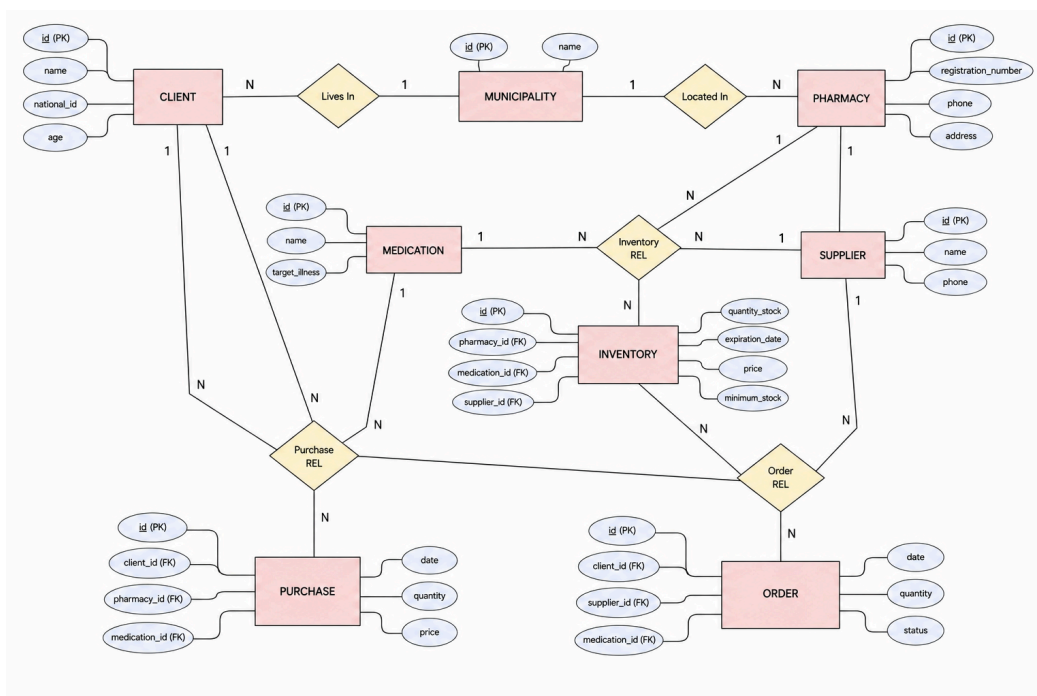


jpa/



4. Model

4.1 Entity relational model



4.2 POJOS Classes

The project includes different pojoes (Plain Old Java Object) classes that represent the entities stored in the database, each class has unique attributes and methods required to manage the information.

Both the Role and User domain POJOs implement the serializable interface. This is a mandatory architectural requirement enforced by JPA. It is essential for managing entities across different transactional states, enabling safe object cloning within memory buffers, and caching detached data without risk of corruption. To make this work safely, these classes include a unique *serialVersionUID*. This number acts like a version control lock for the class. By setting it manually, we make sure that if we change or update the code in the future, the system will still be able to read and open older XML files or backups without getting version errors or crashing the application

User Class :Represents the users of the system and stores authentication information such as username, password and role.

Role Class:Defines the role assigned to each user, such as administrator or pharmacist.

Medication Class: Represents the medications stored in the pharmacy database, including information such as name and target illness.

Pharmacy Class: Represents the pharmacies managed by the application.

Inventory Class: Stores the stock information of medications available in each pharmacy.

Purchase Class: Represents the purchases made by clients in pharmacies.

Municipality Class: Represents the municipalities associated with pharmacies and clients.

Order Class: Represents the orders made by pharmacies to suppliers.

Supplier Class: Stores supplier information such as name and phone number.

Client Class: Represents pharmacy clients and stores personal information.

History Class: Stores historical information and records generated by the application.

GlobalStats Class: Represents global statistical information generated by the system, such as overall pharmacy or inventory data

MedicationStats Class: is focused on storing statistical information related to medications.

4.3 Relationships Between Entities

The entities of the database are connected through different relationships that helps in the functionality of the pharmacy application

1. A **one-to-many** relationships are Municipality and Pharmacy since one municipality can have a bunch of pharmacies, but each pharmacy is located in only one municipality
2. A **one-to-many** relationship is also between Client and Purchase, since one client can make several purchases. On another hand Supplier and Order is has the same type of relationship
3. A **many-to-many** relationship also appears among Pharmacy and Medication due to the fact that pharmacies can stored multiples medications and at the same time medication can exist in different pharmacies, this relationship was implemented through Inventory
4. A **many-to-many** relationship is Client, Pharmacy and Medication through Purchase. A client can purchase different medications from different pharmacies, while a pharmacy can sell medications to many clients.
5. A **many-to-many** relationship includes Supplier, Pharmacy and Medications, since a pharmacy can order many medications from different suppliers, and suppliers can provide medications to several pharmacies through Order.

To sum up, the database contains several N:M relationships that are implemented through intermediate entities that also has its one attribute and in this project: Inventory, Purchase and Order act as it or also called ternary relationship, connecting multiple entities.

4.4 Role System

The database uses a role-based authentication system, which means it can control which users can interact with the application and includes two main roles:

- **Administrator**
- **Pharmacist**

The admin has access to the main functionalities such as creating users, managing pharmacies and controlling system data. On the other hand, the pharmacist role is mainly focused on functionalities related to managing medications, updating inventory information, register purchases, consult stock availability and work with supplier orders.

This system is implemented through the User and Role classes using JPA relationships

4.5 Authentication Subsystem

The application includes an authentication subsystem that is done using JPA which manages user login and role verification.

Firstly, a user must enter a username and a password to access the application. The system validates the information and determines the role associated with the user, allowing access to different functionalities depending on whether the user is an administrator or a pharmacist.

This system is implemented through the User, Role and JPAUserManager classes. Passwords are encrypted using BCrypt before being stored in the database, so it can maximize the security of the application.

5. Database Managers

This section provides an architectural explanation of the data access and persistence layer implemented in the Pharmacy Inventory System.

5.1 ConnectionManager and SQLManager

The ConnectionManager class acts as a centralized database utility responsible for initializing the database driver, establishing the link to the storage file, and enforcing relational constraints at the database level.

SQLManager implements the business logic and stores the raw SQL string statements. The structural bridge between both components is established via Dependency Injection. The SQLManager cannot interact with the database without an open transactional context. This hierarchy ensures that all SQL execution units reuse a single, unified database

connection context, preventing resource waste and keeping database setup separated from entity queries.

Also, the ConnectionManager class is related with the DatabaseInitialization class because both are involved in the database initialization process. The DatabaseInitialization class receives the database connection and is responsible for creating the database tables and inserting a small set of initial sample data. The initial data is inserted using INSERT OR IGNORE statements, so duplicated data is avoided when the program is executed more than once.

Finally, ConnectionManager creates an instance of DatabaseInitializer and calls its main methods.

This separation makes the code cleaner, because ConnectionManager handles the connection logic, while DatabaseInitializer handles the database setup logic.

5.2 JDBC Managers and Main Functions

The core data access layer of the application is built entirely upon the Data Access Object (DAO) pattern.

All JDBCManagers share the exact same structural design, behavior and technical workflow. They need a constructor object to connect them to the database. All operations that involve dynamic user variables use *PreparedStatement* to ensure that data parameters are safely compiled before execution.

Every database transaction is wrapped inside *try-catch* blocks to prevent application crashes by catching runtime database drops.

- JDBCMedicationManager - governs the central pharmaceutical catalog by adding, searching, deleting...
- JDBCInventoryManager - controls store warehouses, quantities and stock safely limits
- JDBCOrdersManager - manages batch supply requests generating supply order or updating order processing states
- JDBCPharmacyManager - coordinates physical pharmacy branch locations and licensing
- JBDCPurchaseManager - records client purchases, links transaction timestamps, and calculates checkout totals
- JDBCSupplierManager - maintains structural directories of verified distributors

- JDBCHistoryManager - integrates timelines combining incoming and outgoing warehouse stock flows. Merges sales and restocks using unifies *UNION ALL* statements sorted by date
- JDBCReportManager - processes financial analytics, sales stats, and margins

5.3 JPA Managers

Administrative authentication, security layer, and authorization parameters are decoupled into an Object-Relational Mapping design via JPA (Java Persistence API). This layer allows the system to manage security contexts.

The JPAManager class acts as a central coordinator for the entire security system, Instead of exposing individual manager instances globally across the user interfaces, it encapsulates them into a single point access. When the application terminates, calling *disconnect()* ensures a safe cascading cleanup.

The JPARoleManager implements the RoleManager structural interfaces, managing the persistence lifecycles of the Role entity.

The JPAUserManager implements the UserManager interface, managing operator registration, credential validation routines, and security data mapping.

The decision to use JPA only for users and roles is a practical choice to keep the security system completely separate from the daily pharmacy operations. Managing user accounts, logins, and passwords through JPA makes sense because these tasks are straightforward and benefit from the automatic safety features and encryption tools that the framework provides. On the other hand, the core pharmacy features require very complex SQL queries. Using JDBC for the pharmacy business gives us total control over the database speed and allows us to write custom SQL queries easily, creating a balanced system that uses the best tool for each specific job.

5.4 XML Managers

XMLManager class leverages JAXB (Java Architecture for XML Binding) to handle data portability and external file backups.

The manager is structured to handle three main operational scopes, all using the same pipeline concept: Marshalling (converting Java objects into clean, readable XML text files) and Unmarshalling (reading XML files back into operational Java lists).

Since JAXB cannot directly marshal standard Java List interfaces due to the lack of a mandatory XML root element, the system uses specific wrapper classes: *MedicationList*,

SupplierList, and *PharmacyWrapper*. These classes act as structural root containers that encapsulate the raw Java collections, providing the necessary XML annotations (like `@XmlRootElement`) so JAXB can successfully map the object graphs into a valid hierarchical text structure.

The project also references a *pharmacy_system.dtd* template. This file defines the basic layout and rules (like element ordering and attributes) for the pharmacy XML documents.

6. Exceptions and Validations

The system achieves structural stability by relying entirely on Java's native exception framework and defensive input filtering instead of creating redundant custom exceptions. At the database level, raw JDBC connections capture `SQLException` to prevent interface crashes, while the JPA security managers utilize transactional rollback routines (`rollback()`) to shield the database from corrupted or incomplete entries. Simultaneously, the user interface layer operates as a front-line filter that rejects empty fields or negative numbers before data transmission. Finally, the integration layer incorporates strict `.xsd` schema verification within the `XMLManager` to validate incoming external files, completely blocking malformed or out-of-order data structures from entering the application ecosystem.

7. User Interface

The User Interface (UI) of the Pharmacy Inventory System is implemented as a Console-Based Interface (CLI). The interface acts as the primary orchestrator for user commands, capturing input, validating data, and delegating data persistence and retrieval tasks to the respective backend Manager classes (JDBC, JPA, and XML managers).

7.1 Main.java

Purpose and Scope

The `Main.java` class serves as the principal entry point and execution environment for the application. Its primary responsibility is to establish connections to the underlying data sources, authenticate users, and route them to their respective role-based workspaces.

System Initialization

Upon execution, the main method performs critical startup procedures:

Manager Initialization:

The `initializeManagers()` method is invoked to instantiate the central `ConnectionManager` alongside all specific data access objects (e.g., `JDBCMedicationManager`, `JPAUserManager`, `XMLManager`). This ensures that database connections are established and ready before any user interaction occurs.

System Integrity Checks:

The `createInitialAdmin()` method acts as a safeguard. It queries the database using JPA to verify the existence of essential structural components, specifically the admin and pharmacist roles, as well as a default administrative user. If these are absent (such as during a first-time setup), the system automatically provisions them to prevent lockout scenarios.

Authentication Workflow

The initial user interface presents a simple loop offering Login or Exit operations. When the login flow is triggered via `loginMenu()`:

The system captures the Username and Password via standard I/O streams using a `BufferedReader`.

These credentials are passed to the `userManager.login()` method, which verifies them against the securely stored records in the database.

Role-Based Routing:

Upon successful authentication, the system evaluates the Role entity associated with the logged-in user. Depending on whether the role is resolved as "admin" or "pharmacist", the system dynamically transfers control to either the `adminMenu()` or `pharmacistMenu()`, respectively. If authentication fails, or an unknown role is detected, access is denied, and the user is returned to the initial prompt.

Error Handling and Input Validation

The UI layer implements comprehensive exception handling. A global try-catch block surrounds the main loop to prevent unexpected runtime crashes, outputting standardized error messages. Furthermore, specialized utility methods (such as `readInt()`, `readDouble()`, and `readBoolean()`) abstract the complexity of parsing user input, gracefully handling `NumberFormatException` and `IOException` occurrences to ensure continuous execution despite user input errors.

7.2 Administrator Menu

The Administrator Menu (adminMenu()) provides unrestricted, high-level access to the entirety of the Pharmacy Inventory System. It is designed for system administrators who require comprehensive Create, Read, Update, and Delete (CRUD) capabilities across all system modules, including user management and data migration.

Workflow and Subsystems

The menu is structured as a continuous loop governed by a switch-case control mechanism. It presents 14 distinct options, categorized logically into primary entities and sub-menus:

Core Entity Management (Medications): Direct options exist to list, search, add, and delete medications. Adding a medication triggers the UI to sequentially prompt for all required attributes (ID, name, illness, social security coverage, prescription requirements) before encapsulating the data into a Medication POJO and passing it to medicationManager.addMedication().

Operational Sub-Menus (Inventory, Sales, Orders, History, Pharmacy): To prevent visual clutter, complex workflows are nested within sub-menus. For instance, selecting the Inventory menu opens a new contextual loop allowing the admin to find stock, modify quantities, or update sensitive pricing and supplier information.

Administrative Sub-Menus (User Management, XML, Reports): These are exclusive to the Administrator role.

User Management: Allows the creation of new Pharmacist accounts, password resets, and user deletion. It enforces security by verifying username availability and automatically assigning the correct JPA role before database insertion.

XML Menu: Facilitates data portability. It provides options to export or import the entire database schema or specific tables (Medications, Suppliers) using JAXB, alongside a validation mechanism against XSD schemas.

Reports: Triggers the JDBCReportManager to generate analytical business intelligence reports.

Database Interaction

Every action within this menu acts as a direct trigger to the DAO layer. The UI is strictly responsible for capturing parameters; it relies entirely on the boolean returns of the manager classes to determine success or failure, subsequently displaying the appropriate confirmation or error notifications to the user.

7.3 Pharmacist Menu

The Pharmacist Menu (`pharmacistMenu()`) is a streamlined, role-restricted interface tailored for day-to-day pharmacy operations. It focuses on inventory management, sales processing, and stock monitoring while strictly prohibiting structural modifications to the database or administrative user management.

Workflow and Role-Based Restrictions

Similar to the Administrator Menu, it operates on a while loop with a switch-case structure, but offers a reduced set of 9 options. The logic flow is specifically designed to enforce security and operational boundaries:

Read-Only Access to Entities: While the pharmacist can view all medications and search for specific drug information, the options to Add or Delete medications are entirely removed from the interface.

Operational Focus: The core workflow revolves around the Sell medication option and the Inventory menu. During a sale, the UI collects transaction details (Purchase ID, Client, Quantity, Date) and utilizes the `purchaseManager` to execute the transaction, which internally handles stock reduction.

Context-Aware Sub-Menus: The system reuses code smartly while maintaining security. When the pharmacist accesses the Inventory menu, the method `inventoryMenu(boolean adminAccess)` is called with the parameter set to false. This boolean flag dynamically hides administrative options (like modifying a medication's base price or supplier) and restricts the pharmacist strictly to finding records and manually adjusting stock quantities (increase/reduce).

Alerting and History: The pharmacist retains full access to the Stock alerts menu to proactively manage low or depleted stock, and the History menu to trace the operational movements of inventory across different pharmacy branches.

User Guidance

The interface guides the pharmacist sequentially. For example, if a user attempts to sell a quantity less than or equal to zero, the UI immediately intercepts the input, throws a validation warning ("Quantity must be greater than 0."), and aborts the database call, thus maintaining database integrity without throwing a fatal exception.

8. Testing and Validation

Testing focused on confirming that tables initialize without errors, data updates properly, logins remain secure, and external files import safely.

8.1 CRUD Testing

The core business operations (Create, Read, Update, Delete) managed by the JDBC repository layers were tested extensively using a black-box testing approach combined with direct database inspections.

Multiple medications and suppliers were registered and updated through the application menus. Testing confirmed that the system correctly generated primary keys, modified existing attributes (such as stock levels or phone numbers) without creating duplicates, and successfully removed target records. Every database change was verified externally using exploration tools, proving that relational integrity and foreign key constraints work perfectly across all tables.

8.2 Login Testing

The authentication system was tested using both valid and invalid credentials. Entering a correct username and password successfully triggered *BCrypt.checkpw()*, retrieved the user's Role, and opened the correct dashboard view. Conversely, entering incorrect passwords or non-existent usernames resulted in a safe access denial without any system crashes.

8.3 XML Testing

Data export and import loops were verified by saving medication and supplier lists into .xml files, wiping the database tables, and successfully restoring the data via JAXB. Additionally, when a file with intentional structure errors was processed, the *validateXML* method successfully caught the issues against the .xsd rules and blocked the corrupted data from entering the system.

9. Difficulties and Solved Problems

This section details the main technical roadblocks encountered during the project and how the team resolved them.

9.1 JDBC Problems

One of the earliest difficulties with JDBC was keeping strict alignment between the database columns, the repository interfaces, and the POJO attributes. Because JDBC uses raw SQL strings, any slight typo in an attribute name or column identifier across the interface methods resulted in silent runtime crashes or mapping failures.

The solution was to establish a strict naming convention. We manually cross-referenced every interface query argument with the exact database schemas created in *TestDataBaseCreation*, ensuring that entity variables perfectly matched the relational structure.

9.2 JPA Problems

During the JPA security layer design, we faced an architectural debate: whether to implement a single, monolithic *JPAUserManager* that handled everything, or separate the concerns into a distinct *JPARoleManager* and *JPAUserManager*. Combining them seemed easier at first, but it quickly led to messy, clustered code when handling complex role dependencies. So we chose to split them into independent managers.

9.3 BCrypt Problems

Implementing password hashing required using an external security library (jBCrypt). Initially, when team members cloned the repository from GitHub, the project broke because the external .jar dependency was only configured locally on one machine. To ensure seamless teamwork and version control, we created a dedicated lib folder inside the root project directory, placed the libraries inside it, and committed it to GitHub. This allowed every team member to automatically see, import, and link the library path right after fetching the latest repository changes.

9.4 XML Problems

The main challenge with JAXB was understanding why we could not just marshal a standard Java List of objects directly into an XML file. JAXB requires a single root element tag to anchor the data hierarchy, which a flat database query list cannot provide. After researching JAXB limitations, we realized we needed intermediate container components. We designed the PharmacyWrapper (and specific list wrappers) to act as the global root objects, safely enveloping the raw collections so JAXB could successfully translate our active memory data into valid XML files.

10. Bibliography and Conclusions

This project made it possible to develop a complete pharmacy management system using Java, SQLite, JDBC, JPA and XML.

The project also helped to better understand how JDBC and JPA work, how to organize a project using interfaces and managers, and how different parts of the system interact with each other.

Overall, this practice was useful to improve both Java programming and database management skills.

[GeeksforGeeks | Your All-in-One Learning Portal](#)

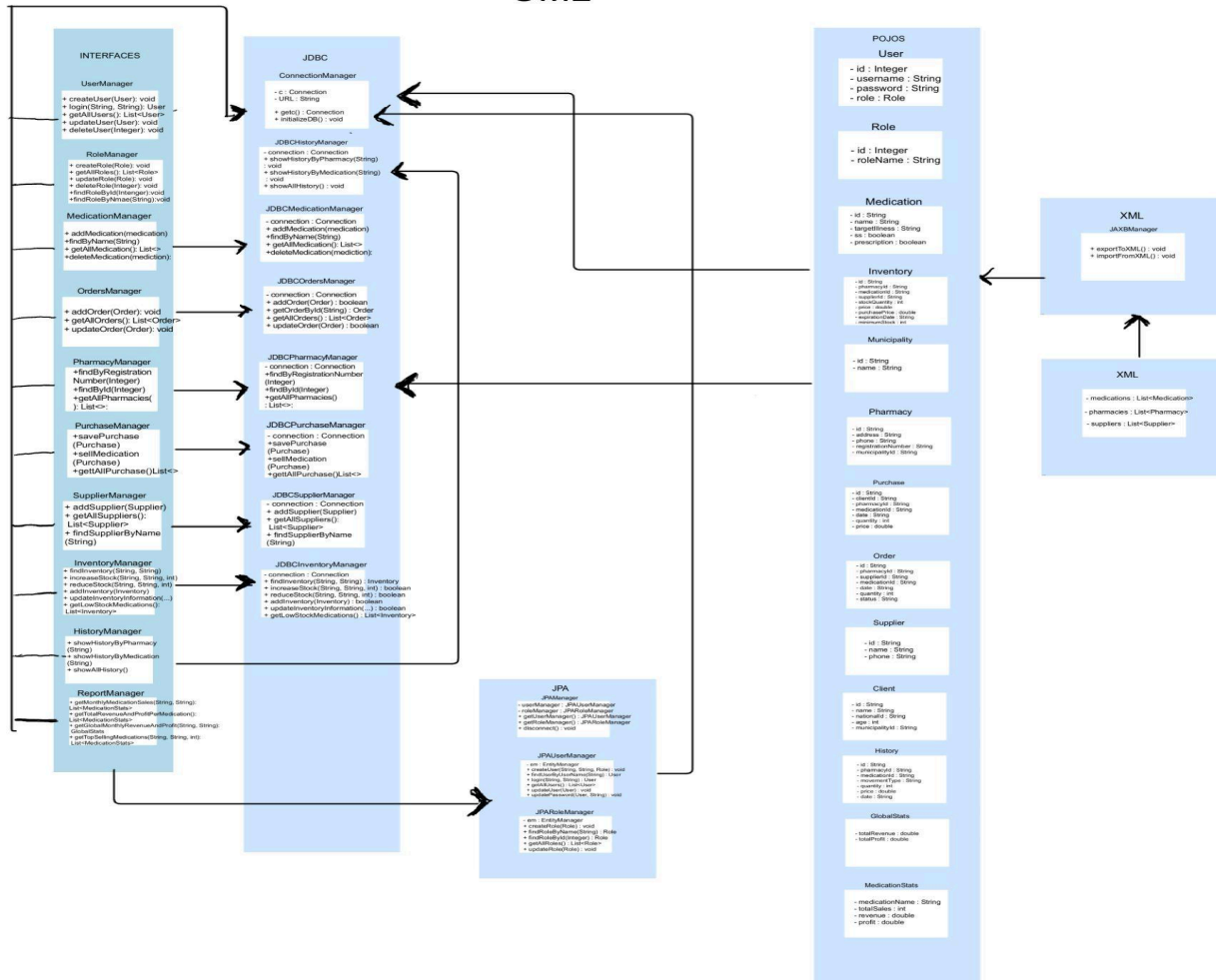
[Java Documentation - Get Started](#)

[ERDPlus](#)

11.UML

11.1 Class diagram and Package Relationship

UML



El JPA se usa para el **login subsystem**, es decir, usuarios y roles. El resto de entidades principales de la farmacia pueden seguir con JDBC.

En userManager y roleManager (interfaces) están las operaciones minimas que se piden en JPA (create, read/search, update and delete).

Siguiente paso: en User.java y Role.java escribir la relación de ASOCIACIÓN obligatoria para el login

Siguiente paso: hacer JPAManager (paola), JPAUserManager (andrea) y JPARoleManager

Por último, hacer un TestJPA (alba) para probar los managers.

Client hereda de user – el cliente es un usuario – El objeto **Client** ya tiene dentro su **username**, su **password** y su **role**. Cuando un cliente hace login, la base de datos ya sabe quién es y qué permisos tiene de una sola vez

IMPORTANTE

En JDBC creamos las tablas a mano en SQL.

En JPA definimos las clases Java como entidades, y JPA puede crear las tablas automáticamente a partir de esas clases.

En el main:

Hay un unico login (username + password). Despues, JPA devuelve el USER completo y el main mira loggedUsrer.getRole().getRoleName()

-> Segun el rol que devuelva puede haber 2 menus: 1) adminMenu() 2) pharmacistMenu()

Admin menu:

1. Show all medications
2. Search medication
3. Add new medication
4. Delete medication
5. Update medication information
6. Increase stock
7. Reduce stock / sell medication
8. Check low stock alerts
9. Create order to supplier
10. Show movement history
11. XML menu
12. User management
13. Logout

Pharmacist menu:

1. Show all medications
2. Search medication
3. Increase stock
4. Reduce stock / sell medication
5. Check low stock alerts
6. Create order to supplier
7. Show movement history
8. Logout